

I2C PROTOCOL

UNDERSTANDING THE I2C INTERFACE BASICS AND MODES



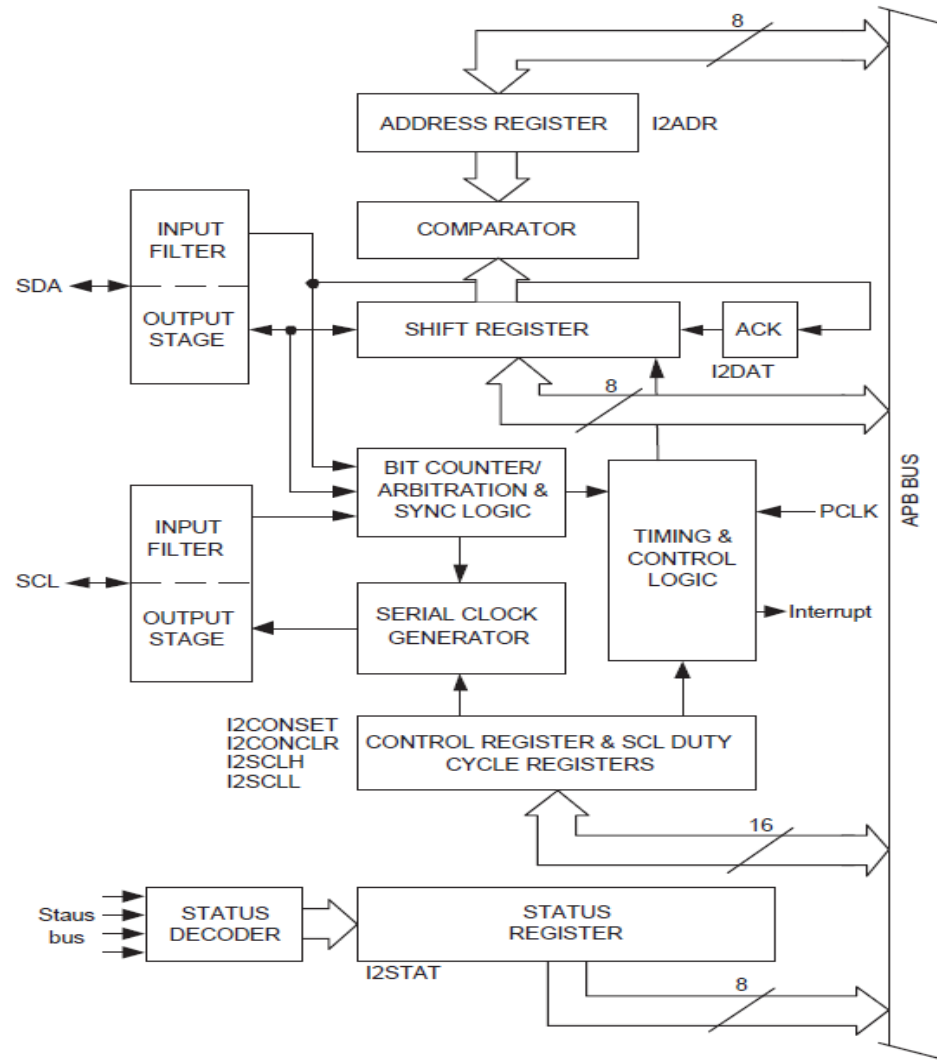
WHAT IS I2C?

- I2C = Inter-Integrated Circuit
- Uses two lines: SDA (Data), SCL (Clock)
- Supports multiple masters and slaves
- Designed for short-distance communication

FEATURES OF I2C0 INTERFACE

- Can be Master, Slave, or both
- Supports arbitration between masters
- Programmable clock rates
- Serial clock synchronization and handshaking
- Supports diagnostic modes

I2C IMPLEMENTATION



REGISTER

11.7 Register description

Each I²C interface contains 7 registers as shown in [Table 135](#) below.

Table 135: I²C register map

Name	Description	Access	Reset value ^[1]	I ² C0 Address and Name	I ² C1 Address and Name
I2CONSET	I²C Control Set Register. When a one is written to a bit of this register, the corresponding bit in the I ² C control register is set. Writing a zero has no effect on the corresponding bit in the I ² C control register.	R/W	0x00	0xE001 C000 I2C0CONSET	0xE005 C000 I2C1CONSET
I2STAT	I²C Status Register. During I ² C operation, this register provides detailed status codes that allow software to determine the next action needed.	RO	0xF8	0xE001 C004 I2C0STAT	0xE005 C004 I2C1STAT
I2DAT	I²C Data Register. During master or slave transmit mode, data to be transmitted is written to this register. During master or slave receive mode, data that has been received may be read from this register.	R/W	0x00	0xE001 C008 I2C0DAT	0xE005 C008 I2C1DAT
I2ADR	I²C Slave Address Register. Contains the 7-bit slave address for operation of the I ² C interface in slave mode, and is not used in master mode. The least significant bit determines whether a slave responds to the general call address.	R/W	0x00	0xE001 C00C I2C0ADR	0xE005 C00C I2C1ADR
I2SCLH	SCH Duty Cycle Register High Half Word. Determines the high time of the I ² C clock.	R/W	0x04	0xE001 C010 I2C0SCLH	0xE005 C010 I2C1SCLH
I2SCLL	SCL Duty Cycle Register Low Half Word. Determines the low time of the I ² C clock. I2nSCLL and I2nSCLH together determine the clock frequency generated by an I ² C master and certain times used in slave mode.	R/W	0x04	0xE001 C014 I2C0SCLL	0xE005 C014 I2C1SCLL
I2CONCLR	I²C Control Clear Register. When a one is written to a bit of this register, the corresponding bit in the I ² C control register is cleared. Writing a zero has no effect on the corresponding bit in the I ² C control register.	WO	NA	0xE001 C018 I2C0CONCLR	0xE005 C018 I2C1CONCLR

[1] Reset value reflects the data stored in used bits only. It does not include reserved bits content.

MASTER TRANSMITTER MODE

- Master sends START condition
- Sends Slave address + Write bit
- Transfers data bytes
- Ends with STOP or repeated START

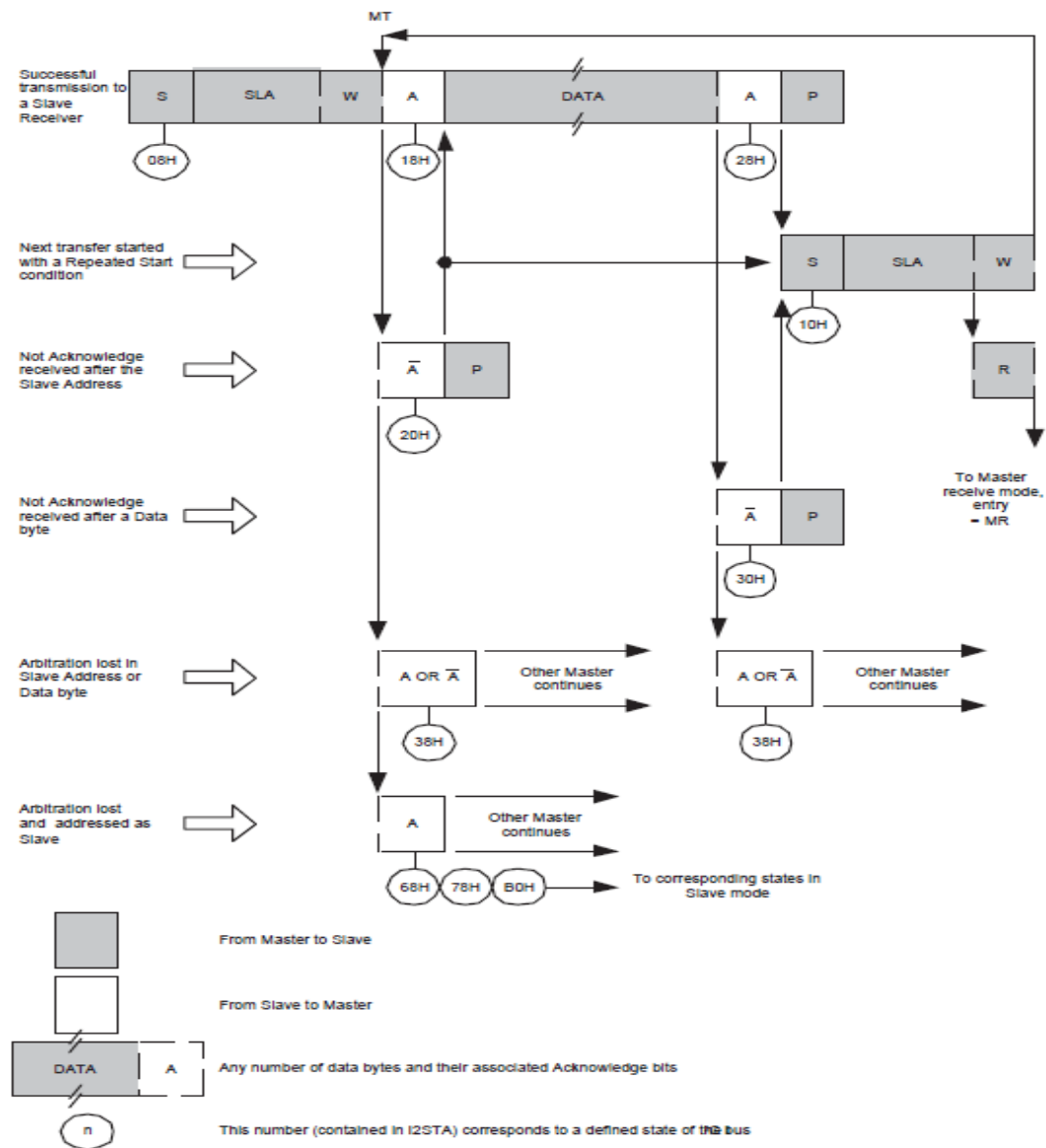


Fig 33. Format and States in the Master Transmitter mode

MASTER RECEIVER MODE

- Master sends START condition
- Sends Slave address + Read bit
- Receives data from Slave
- Sends ACK/NAK accordingly

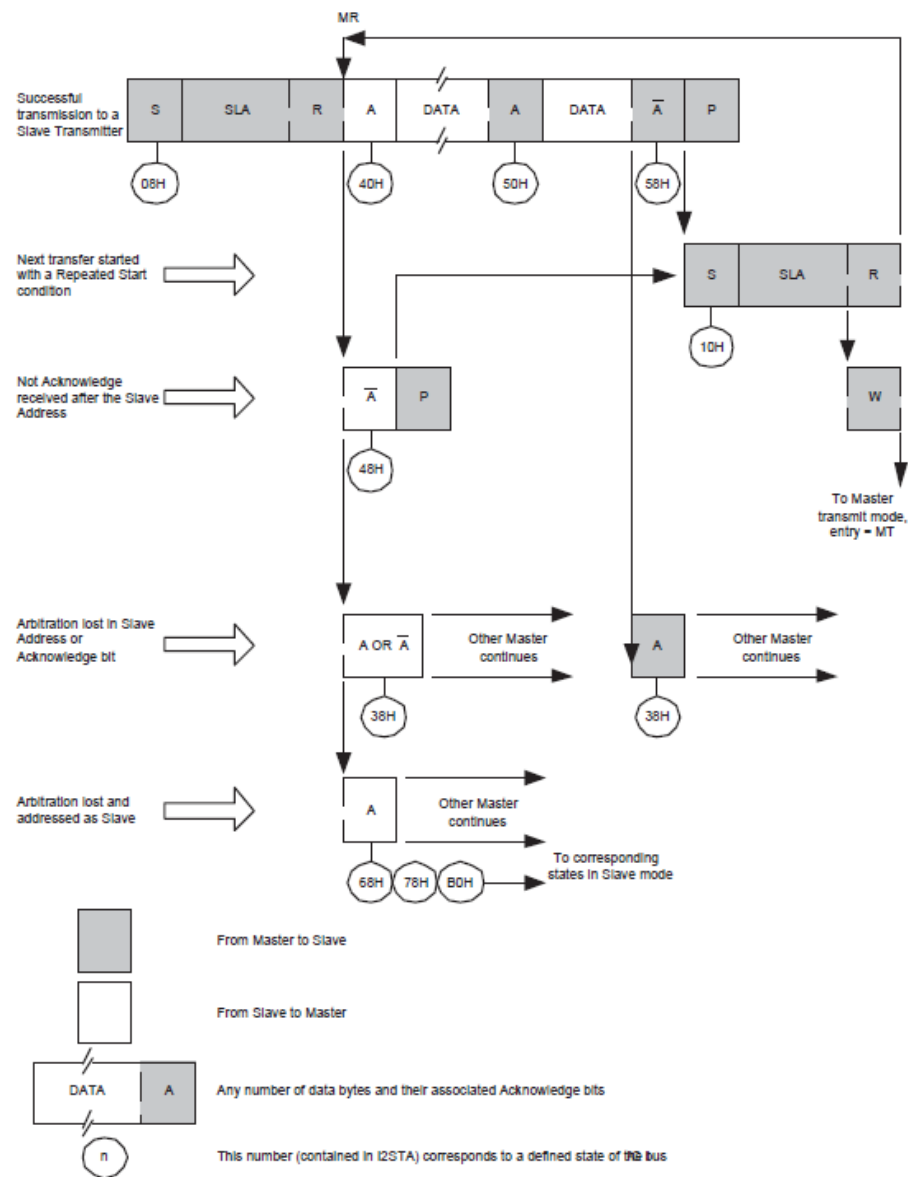


Fig 34. Format and States in the Master Receiver mode

SLAVE RECEIVER MODE

- Slave waits for address match
- Receives data from Master
- Sends ACK or NAK

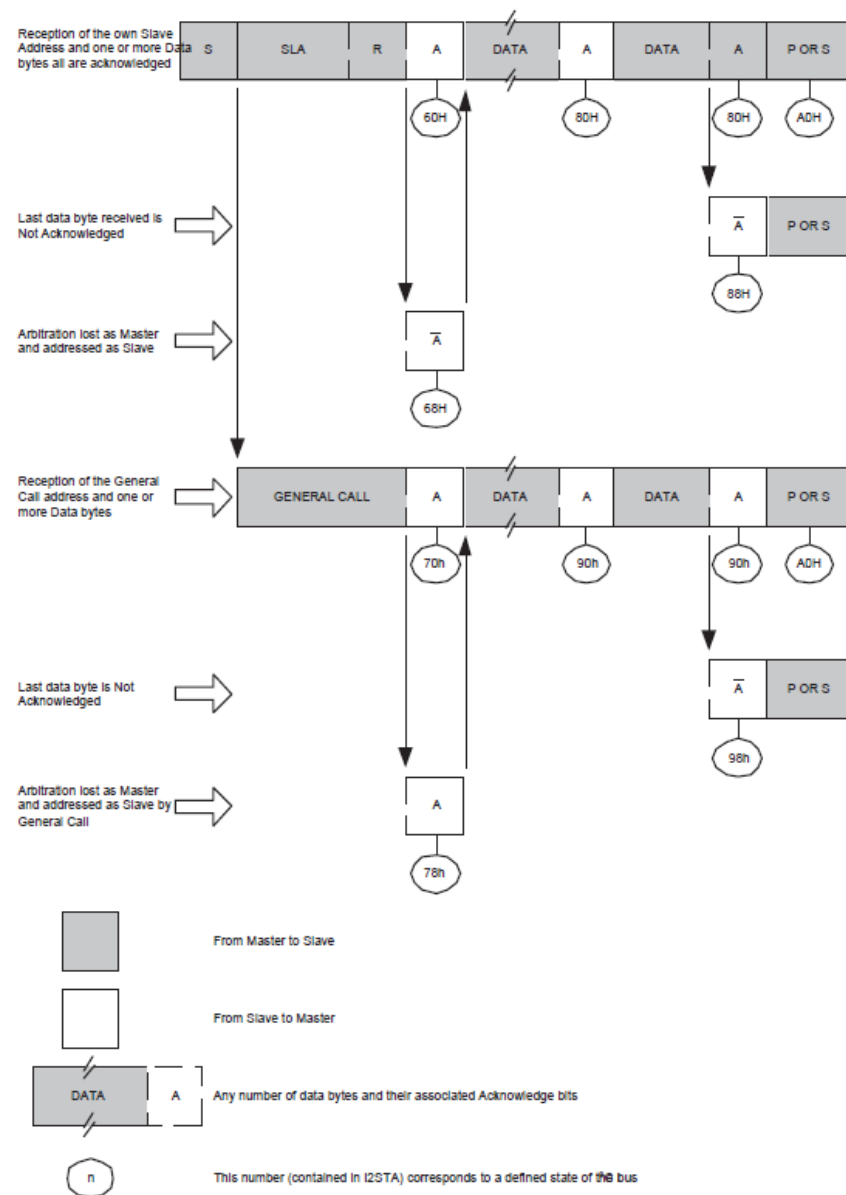


Fig 35. Format and States in the Slave Receiver mode

SLAVE TRANSMITTER MODE

- Slave waits for address with Read bit
- Sends data to Master
- Waits for ACK/NAK
- May switch back to not-addressed state

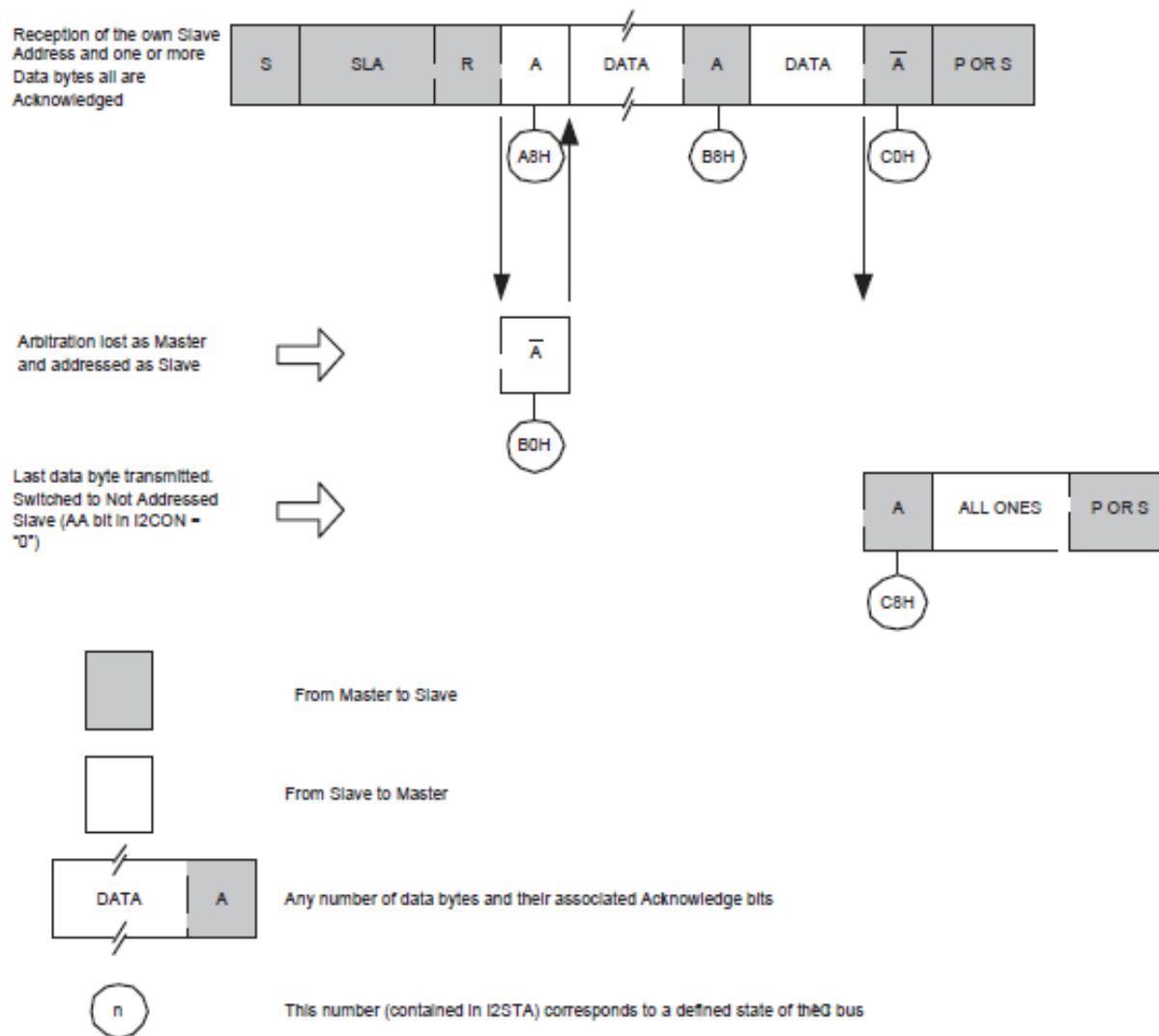


Fig 36. Format and States in the Slave Transmitter mode

CODE

```
• #define SLAVE_ADDR 0x50
• void I2C_Init() {
•     I2C_Enable();    // Enable
      I2C
•     I2C_SetClock(100000); // Set
      100kHz clock
• }
• void I2C_Write(uint8_t data) {
•     I2C_Start();
•     I2C_SendByte(SLAVE_ADDR
      << 1); // SLA+W
•     I2C_WaitAck();
•     I2C_SendByte(data);
•     I2C_WaitAck();
•     I2C_Stop();
• }
• uint8_t I2C_Read() {
•     uint8_t val;
•     I2C_Start();
•     I2C_SendByte((SLAVE_ADDR <<
      1) | 1); // SLA+R
•     I2C_WaitAck();
•     val = I2C_ReceiveByte();
•     I2C_SendNACK();
•     I2C_Stop();
•     return val;}
```


THANK YOU